

Deep research briefing on developing Claude Skills for GitHub publication under CC-BY

Skills architecture and platform surfaces

Anthropic's "Skills" (often referred to as "Agent Skills") are modular, filesystem-based capability packages: a Skill is a directory whose entrypoint is a `SKILL.md` file containing YAML frontmatter (metadata) plus Markdown instructions, with optional companion resources (e.g., templates, reference docs) and executable code (scripts). ¹

A central design goal is **progressive disclosure**: Claude preloads only lightweight metadata (not whole instructions) so you can have many installed Skills without paying a large context penalty. When the model decides a Skill is relevant, it loads `SKILL.md`; it can then selectively read other linked files or run scripts only when needed. ² This "skills as onboarding guides" metaphor is explicit in Anthropic's engineering write-up, which stresses organising skill content so Claude can navigate to detail as required rather than flooding the context window up front. ³

Claude Skills exist across multiple "surfaces", each with distinct distribution and runtime constraints:

Claude.ai supports pre-built Skills (working "behind the scenes" for document creation) and user-uploaded custom Skills (zip upload via settings), but custom Skills on Claude.ai are **per-user** rather than centrally managed. ⁴

Claude Code supports **custom Skills** as local or project files, with automatic discovery and optional plugin-based distribution. Claude Code explicitly aligns with the cross-platform **Agent Skills open standard**, while also adding Claude-Code-specific extensions (invocation control, subagent execution, dynamic context injection). ⁵

The Claude API supports both **Anthropic-managed Skills** (e.g., `pptx`, `xlsx`, `docx`, `pdf`) and your **workspace-private custom Skills** uploaded via `/v1/skills`. Both are attached to a request via the code execution container and share the same container shape; the key difference is source and lifecycle management. ⁶

Claude Agent SDK supports filesystem-based Skills via `.claude/skills/` when the Skill tool is enabled in `allowed_tools`. ⁷

A frequent point of confusion is "portability" versus "sync": the **format** is intended to work across Claude products (and increasingly other vendors), but **custom Skills do not automatically sync across surfaces**. If you want the same Skill in Claude.ai and the API, you must upload/manage it separately per surface. ⁸

Release notes indicate that by late 2025 Skills were positioned as part of a broader ecosystem, including an open standard ("Agent Skills") and partner-built Skills discovery/distribution mechanisms. ⁹

Authoring Skills that trigger reliably in real workflows

Metadata is the primary “routing layer”

In practice, the `description` field is the dominant signal the model uses to decide whether to load a Skill. Anthropic’s authoring guidance emphasises using the description to state both *what the Skill does* and *when it should be used*, with concrete trigger terms and contexts. ¹⁰ The same best-practices guidance warns about viewpoint inconsistency and explicitly recommends writing descriptions in **third person** to avoid discovery problems introduced by system-prompt injection of the description. ¹¹

Naming is not cosmetic: Anthropic recommends consistent naming patterns (often **gerund form**, verb + “-ing”) to make Skills easier to reference and manage as the library grows. ¹¹

The open specification at agentskills.io formalises constraints (e.g., max lengths and permitted characters). Notably, it also requires that the `name` match the parent directory name, which is important if you want cross-tool interoperability without shims. ¹²

Skill content types and invocation control

Claude Code documentation distinguishes between “reference content” (domain conventions/patterns that should apply inline) and “task content” (step-by-step operational runbooks you expect to call explicitly, often with side effects). For task-like Skills, Claude Code recommends preventing automatic triggering using `disable-model-invocation: true` so Claude does not autonomously run a deploy/commit/send workflow merely because the conversation context resembles completion. ¹³

Claude Code provides two key invocation gates:

- `disable-model-invocation: true` (manual-only: you can invoke, Claude cannot) and
- `user-invocable: false` (model-only: Claude can invoke, but it is hidden from the user’s slash menu). ¹⁴

Importantly, the invocation gate affects **what is loaded into context**: in Claude Code, when `disable-model-invocation` is set, the Skill description is removed from the model’s context entirely (a useful security and predictability lever). ¹⁵

Tooling boundaries, least privilege, and safe composability

Claude Code supports an `allowed-tools` frontmatter field to constrain the tools that can be used when a Skill is active (e.g., `Read`, `Grep`, `Glob` for a read-only exploration Skill). ¹⁶ It also documents permission rules for allowing/denying the Skill tool and individual Skills by name (exact or prefix match); this underpins a “capability-based” approach where risky Skills are denied or require interactive approval. ¹⁷

On the API surface, **environment constraints are stricter**: Skills run in the code execution container with **no network access** and **no runtime package installation**, so authoring should avoid assuming downloads or pip installs at execution time; depend only on preconfigured packages. ¹⁸ On Claude Code (local machine), network access is effectively whatever the user machine has, and Anthropic discourages global package installation that could pollute the developer environment. ¹⁹

Arguments and dynamic context injection as “workflow adaptors”

Claude Code Skills can accept arguments via `/skill-name ...`; those arguments are available to the prompt through placeholders such as `$ARGUMENTS`, `$ARGUMENTS[N]`, and `$N`. ²⁰

For highly technical workflows, the most powerful “bridge” from your current toolchain into a Skill is **dynamic context injection**: Claude Code supports a preprocessing syntax that runs shell commands first and replaces placeholders with their output before the prompt is sent to Claude. This enables Skills that deterministically collect repo state (PR diffs, file lists, logs) rather than relying on the model to remember to run commands. ²¹

Practical dos and don’ts grounded in official guidance

Do treat `SKILL.md` as a navigational entrypoint, not a dumping ground. Claude Code explicitly advises keeping `SKILL.md` under ~500 lines and moving detailed material into separate referenced files, so Claude can load detail only when needed. ²²

Do keep your always-on project memory (e.g., `CLAUDE.md`) lean and push “sometimes relevant” workflows into Skills. Anthropic warns that bloated `CLAUDE.md` files lead Claude to ignore important instructions and recommends Skills for domain workflows that are only occasionally relevant. ²³

Don’t rely on untrusted Skills without auditing. Anthropic’s Skills overview explicitly frames Skills as equivalent to installing software: scripts and instructions can be malicious, including data exfiltration and tool misuse; external URL fetches are highlighted as an additional risk (dependencies can change over time). ²⁴

Do explicitly model security review for enterprise-grade workflows. Anthropic’s enterprise guidance provides a risk tier assessment and a concrete review checklist (code execution, network calls, hardcoded credentials, broad filesystem scope, instruction manipulation, MCP references). ²⁵

Don’t assume Creative-Commons-style licensing decisions are “purely documentation”: if your Skill bundles scripts, you are publishing software as well as text; this materially affects downstream reuse and compliance (discussed in the release section).

File and repository structures for production-grade Skill libraries

Skill-internal file organisation and “how many files per Skill”

At minimum, a Skill is a directory with a required `SKILL.md` entrypoint; additional files are optional and should be justified by progressive disclosure or determinism (scripts). ²⁶

From Claude Code’s documentation perspective, a “well-structured” Skill commonly looks like:

```
my-skill/
├── SKILL.md           # required; overview + when/how to use + pointers
├── reference.md       # optional; deep reference, loaded when needed
├── examples/         # optional; “what good looks like”
│   └── sample.md
```

```
└─ scripts/           # optional; deterministic automation
   └─ validate.sh
```

This pattern is directly shown in Claude Code docs and aligns with the design goal of keeping `SKILL.md` as an overview and navigation hub. ²⁷

Concrete official example (Claude-created Skill used in production): PDF Skill

Anthropic's public Skills repository includes the PDF Skill that underpins Claude's PDF capabilities, and Anthropic's engineering blog uses it as a canonical example of progressive disclosure—`SKILL.md` for workflow overview plus separate, linked files (e.g., for form filling). ²⁸ The PDF Skill frontmatter also demonstrates a `license` field, and the body is organised as a practical handbook (libraries, command-line tools) with explicit references to additional docs for advanced use cases. ²⁹

Concrete official example (developer-facing): Visual-output Skill pattern

Claude Code docs include a “codebase visualiser” Skill that bundles a Python script to generate an interactive HTML report, illustrating a robust “deterministic output generator + human-facing artefact” pattern. This example shows how `allowed-tools` can be scoped to `Bash(python *)` and how your Skill instructions can explicitly drive script execution steps. ³⁰

Repository organisation for multiple Skills

Claude Code enables Skills at multiple scopes, which naturally suggests different repository layouts:

- “Project skills” live in `.claude/skills/<skill-name>/SKILL.md` and are committed to your repository for team sharing. ³¹
- “Personal skills” live in `~/.claude/skills/<skill-name>/SKILL.md` and apply across projects. ³²
- “Plugin skills” live under a plugin's `skills/` directory and are namespaced to avoid conflicts (`plugin-name:skill-name`). ³³
- Enterprise-managed Skills can be deployed org-wide through managed settings (for organisations using that deployment mode). ³⁴

Claude Code also supports automatic Skills discovery from nested `.claude/skills/` directories when working inside subdirectories (useful for monorepos where packages have distinct workflows). ³⁵

One repository with many Skills vs one repository per Skill

The decision is mainly about operational complexity (versioning, CI) versus discoverability and reuse granularity. A highly technical library can succeed in either model, but the “right” choice depends on whether your Skills are intended to be installed together or independently.

Approach	What it looks like	Strengths	Weaknesses	When it tends to win
Multi-skill monorepo	<code>skills/</code> <code><skillA>/</code> , <code>skills/</code> <code><skillB>/</code> (or <code>.claude/</code> <code>skills/...</code> for project-bound)	Shared CI, shared contributor guidelines, shared utilities/templates; easier to keep consistent naming and doc standards across a library	Harder for users to “vendor” exactly one Skill; higher blast radius of repo-wide changes; noisy versioning if Skills evolve at different cadences	When Skills are designed as a cohesive toolkit (“skill suite”), or you want strong governance and uniform quality
One repo per Skill	Each repo contains exactly one Skill directory (plus docs/ tests)	Clean install story, clear versioning, minimal collateral changes; easier GitHub topics/ searchability per Skill	Duplication of templates and CI; governance overhead scales with Skill count	When Skills are independent products, have different maintainers, or target different user segments
Hybrid (recommended for many teams)	One “core” skills repo + optional satellite repos for heavyweight Skills	Balance: central catalogue + specialised repos; reduces duplication for common components	Requires documentation discipline to avoid fragmentation	When you expect “long tail” Skills (experimental) alongside a stable, curated set

A key compatibility constraint: the open spec expects the Skill directory name to match the `name` field, and many tools assume 1 Skill = 1 directory. If you publish multiple Skills in one repo, keep each Skill self-contained and avoid cross-references that break vendoring. ³⁶

Community examples that illustrate “what works”

A recurring pattern in capable community Skills is treating `SKILL.md` as *portable engineering playbooks* rather than ad-hoc prompts. For example, the `better-chatbot-patterns` Skill in a community repository is explicitly framed as a portable template collection you can adapt across projects, which aligns with Anthropic’s guidance to keep Skills composable and reusable rather than tied to a single conversation. ³⁷

Another community Skill example (`frontend-design-fix-svelte`) illustrates a common ecosystem pattern: adding non-standard metadata fields such as `version`, `framework`, and “tools” lists in frontmatter to aid human browsing and catalogue indexing. This can be useful, but it should be treated as **extra metadata** rather than something Claude’s Skill loader is guaranteed to interpret—stick to spec-compliant fields for portability, and consider using an additional machine-readable manifest if you need richer metadata (discussed below). ³⁸

Packaging and distribution mechanics

Required and optional metadata across standards and Claude Code extensions

At minimum, the open Skill format requires `name` and `description` with strict constraints (length, character set), and agentskills.io documents optional fields such as `license` and `compatibility` to encode licensing and environment expectations. ³⁹

Claude Code supports additional frontmatter fields to govern runtime behaviour (e.g., `disable-model-invocation`, `user-invocable`, `allowed-tools`, `context`, `agent`, `hooks`) and uses markdown content substitutions for arguments and session identifiers. ⁴⁰

One nuance: Claude Code can infer `name` from the directory name and can infer `description` from the first paragraph if omitted—convenient for local use, but suboptimal for portability and reliable triggering across toolchains. Including explicit `name` and `description` is the safer cross-platform choice. ⁴¹

Packaging methods by surface

Surface	How you package Skills	Distribution scope	Key constraints you must design for
Claude Code	Folder(s) under <code>~/ .claude/ skills/</code> (personal) or <code>.claude/skills/</code> (project); optional plugin packaging	Personal, team repo, or plugin-based distribution	Local machine execution; supports <code>allowed-tools</code> , subagents (<code>context: fork</code>), dynamic context injection (<code>!</code> shell preprocessor) ⁴²
Claude.ai	Custom Skills uploaded as a zip file via Settings > Features (plans and code execution required)	Per-user only, manual per member	Network access may vary by settings; not centrally managed in Claude.ai ²⁴
Claude API	Upload as files to <code>/v1/ skills</code> (directory helper, zip, or explicit file tuples); attach via <code>container.skills</code>	Workspace-wide for uploaded custom Skills	Max 8MB upload; must include top-level <code>SKILL.md</code> ; Skills run in code execution container with no network and no runtime package installation ⁴³
Claude Agent SDK	Place Skills under <code>.claude/ skills/</code> and enable Skill tool	Depends on your deployment	Same filesystem-centric assumptions; must enable <code>"Skill"</code> in <code>allowed_tools</code> ⁷

For the Claude API specifically, Anthropic provides multiple upload affordances (recommended Python helper `files_from_dir`, zip upload, or explicit tuples), and documents strict requirements such as `SKILL.md` at top level, common root directory, total upload size under 8MB. ⁴⁴

Versioning and lifecycle

The Claude API has **first-class Skill versioning**: Anthropic-managed Skills use date-based versions, while custom Skills use auto-generated epoch timestamps; consumers can pin versions or request "latest".⁴⁵ This is distinct from how you might version a GitHub repository (SemVer tags, releases). If you publish Skills on GitHub under CC-BY, you will likely want **human-oriented repo versioning** and be prepared for API-assigned version identifiers when you upload to the Claude Console/API.⁴⁶

For Claude Code plugin/distribution workflows, Anthropic's own Skills repository demonstrates a marketplace+plugin approach (registering a repo as a marketplace and installing plugins), which is relevant if you want "installable bundles" rather than copy-paste folder installs.⁴⁷

Testing, security, and release engineering for shareable Skill repositories

Testing approaches that map to Skill failure modes

Official guidance repeatedly implies that Skills should be iterated and validated like software rather than treated as static prompt text:

- The Claude Help Centre recommends testing incrementally and starting with simple Markdown before adding scripts.⁴⁸
- Anthropic's enterprise guidance formalises vetting and sandbox testing of scripts, ensuring behaviour matches the stated purpose.²⁵
- Claude Code encourages the use of "allowed-tools" and permission rules to bound what a Skill can do, which can be treated as part of your "testable contract" (least privilege by default).⁴⁹

A testing matrix that works well for technical teams is:

Testing layer	What you validate	How it typically breaks	Practical implementation notes
Spec conformance	YAML parses; name / description constraints; folder name matches name	Silent non-discovery; toolchain incompatibility	Use the agentskills.io specification as the canonical field/constraint reference ¹²
Behavioural triggering	Skill is invoked when it should be; not invoked when it shouldn't	Vague descriptions; conflicting Skill overlap; viewpoint mismatch	Follow Anthropic guidance on specific third-person descriptions with trigger terms ¹⁰
Script determinism	Scripts run in expected environments and produce stable outputs	Missing dependencies; relying on network; OS assumptions	API container cannot access network or install packages; Claude Code runs locally with different constraints ¹⁸

Testing layer	What you validate	How it typically breaks	Practical implementation notes
Safety and policy	Skill does not exfiltrate secrets; no surprising tool calls	Hidden instructions; supply-chain-like behaviours	Apply enterprise review checklist, scan for network calls, broad filesystem access, hardcoded credentials ⁵⁰
Regression	Changes do not degrade outputs across versions	Prompt drift; “helpful” edits breaking step order	Maintain fixture prompts and expected outputs; re-run on each PR (CI)

Although not an Anthropic document, recent security research and incident write-ups underscore that “Skill supply chain” security is non-theoretical: an empirical 2026 study of tens of thousands of Skills reports substantial vulnerability prevalence at scale, and a concrete real-world case shows how hallucinated `npx` commands propagated through Skill repositories can become an attack surface via typosquatting/slopsquatting. These reinforce the need for the vetting guidance Anthropic gives. ⁵¹

Security posture for publishing to GitHub

Anthropic’s baseline recommendation is to treat Skills from third parties as potentially malicious and to audit all bundled files (including scripts and linked content). ²⁴ For a GitHub-published Skill under CC-BY, “auditable by default” should be a release criterion:

Write `SKILL.md` so its intent is unambiguous and consistent with any scripts it triggers, and avoid commands that implicitly download or execute remote code (especially in situations where Claude will auto-approve or a developer might approve once and reuse approvals). ⁵²

Prefer local, deterministic tooling, and if you do need network calls in Claude Code (local machine), document them in a compatibility section and consider restricting tool usage to narrow patterns. ⁵³

Releasing and sharing effectively under CC-BY

Documentation standards that reduce misuse

A high-quality public Skill repo should include, at minimum:

Clear installation instructions per surface (Claude Code copy/install, Claude.ai zip upload, Claude API upload), because Skills do not sync across surfaces and users must repeat setup per target environment. ⁵⁴

Explicit environment constraints and compatibility notes (especially: Claude API no network/no installs), ideally in a `compatibility` field (where supported) or a README section that states tested environments. ⁵⁵

A “security considerations” section that sets expectations about what the Skill reads/writes/executes and which tools it expects (and how to restrict them), aligning with Anthropic’s focus on auditability and risk review. ⁵⁶

Versioning strategy for GitHub distribution

The Claude API auto-generates version identifiers for custom Skills (epoch timestamps) and supports pinning or `"latest"`.⁵⁷ GitHub consumers, however, will expect conventional software versioning. A pragmatic approach is:

Use SemVer tags/releases for repositories (e.g., `v1.2.0`) to describe behavioural changes in the prompt/script bundle, while treating Claude API versions as deployment artefacts (pinned for runtime stability). The API's own versioning model supports this "pin in production, float in dev" pattern.⁵⁷

Licensing considerations for CC-BY

The Agent Skills spec explicitly supports an optional `license` field in Skill metadata, which can reference a license name or a bundled license file—useful for machine-readable signalling when Skills are redistributed via catalogues or installers.⁵⁸

That said, CC-BY is primarily designed for creative works (text/media), whereas Skills often bundle **software** (scripts). Publishing scripts under CC-BY can create friction for downstream developers accustomed to OSI-approved software licenses (and for organisations with standard open-source compliance tooling). If you are committed to CC-BY, a common compromise is dual-licensing: CC-BY for the Markdown guidance and an OSI license for scripts—while still meeting your attribution goals. This is not legal advice; consult counsel for your specific distribution needs.

Also note that if you incorporate third-party code or knowledge into your Skill (including copied templates or code snippets), CC-BY does not override upstream license obligations; you must preserve notices and comply with the terms of included dependencies. Anthropic's own Skills repository explicitly separates open-source Skill content from "source-available, not open source" document Skills, reinforcing that licence boundaries matter in Skill distribution.⁵⁹

Community sharing practices and discovery

The emerging ecosystem has converged around GitHub-first sharing (repos and curated lists), including community "awesome" lists that aggregate known Skills and link to source repositories.⁶⁰ For discoverability, ensure your repo uses consistent Skill naming, includes concise descriptions, and documents installation paths and constraints—these map directly to how Claude (and catalogues) decide what a Skill is for and whether it can run.⁶¹

Finally, if you are publishing a multi-skill repository, strongly consider including a top-level catalogue that lists each Skill directory, its trigger description, supported environments, and tool requirements. This addresses the practical reality that users increasingly want to "vendor just one Skill" into their `.claude/skills/` tree, even when you ship a suite.

¹ ⁸ Introducing Agent Skills | Claude

<https://claude.com/blog/skills>

² ⁴ ⁷ ¹⁹ ²⁴ ²⁶ ⁵³ ⁵⁴ ⁵⁶ Agent Skills - Claude API Docs

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

³ ²⁸ Equipping agents for the real world with Agent Skills \ Anthropic | Claude

<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

5 13 14 15 16 17 20 21 22 27 30 31 32 33 34 35 40 41 42 49 **Extend Claude with skills - Claude Code Docs**

<https://docs.anthropic.com/en/docs/claude-code/skills>

6 18 43 44 45 46 57 **Using Agent Skills with the API - Claude API Docs**

<https://platform.claude.com/docs/en/build-with-claude/skills-guide>

9 **Release Notes | Claude Help Center**

https://docs.anthropic.com/en/release-notes/claude-apps?utm_source=chatgpt.com

10 11 61 **Skill authoring best practices - Claude API Docs**

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>

12 36 39 55 58 **Specification - Agent Skills**

https://agentskills.io/specification?utm_source=chatgpt.com

23 **Best Practices for Claude Code - Claude Code Docs**

https://docs.anthropic.com/en/docs/claude-code/best-practices?utm_source=chatgpt.com

25 50 52 **Skills for enterprise - Claude API Docs**

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/enterprise>

29 **<https://raw.githubusercontent.com/anthropics/skill...>**

https://raw.githubusercontent.com/anthropics/skills/main/skills/pdf/SKILL.md?utm_source=chatgpt.com

37 **<https://raw.githubusercontent.com/secondsky/claude...>**

https://raw.githubusercontent.com/secondsky/claude-skills/main/plugins/better-chatbot-patterns/skills/better-chatbot-patterns/SKILL.md?utm_source=chatgpt.com

38 **<https://raw.githubusercontent.com/matteocervelli/l...>**

https://raw.githubusercontent.com/matteocervelli/lms/main/frontend-design-system/skills/_archive/frontend-design-fix-svelte/SKILL.md?utm_source=chatgpt.com

47 59 **[raw.githubusercontent.com](https://raw.githubusercontent.com/anthropics/skills/main/README.md)**

<https://raw.githubusercontent.com/anthropics/skills/main/README.md>

48 **How to create custom Skills | Claude Help Center**

<https://support.claude.com/en/articles/12512198-how-to-create-custom-skills>

51 **Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale**

https://arxiv.org/abs/2601.10338?utm_source=chatgpt.com

60 **BehiSecc/awesome-claude-skills**

https://github.com/BehiSecc/awesome-claude-skills?utm_source=chatgpt.com